



C--

C-- (pronounced *C minus minus*) is a C-like programming language, designed to be generated mainly by compilers for high-level languages rather than written by human programmers. It was created by functional programming researchers Simon Peyton Jones and Norman Ramsey. Unlike many other intermediate languages, it is represented in plain ASCII text, not bytecode or another binary format.^{[1][2]}

There are two main branches:

- C--, the original branch, with the final version 2.0 released in May 2005^[3]
- Cmm, the fork actively used as the intermediate representation (IR) in the Glasgow Haskell Compiler (GHC)^{[4][5]}

Design

C-- is a "portable assembly language",^[6] designed to ease the implementation of compilers that produce high-quality machine code.^[7] This is done by delegating low-level code-generation and program optimization to a C-- compiler. The language's syntax borrows heavily from C while omitting or changing standard C features such as variadic functions, pointer syntax, and aspects of C's type system, because they hamper essential features of C-- and ease of code-generation.

The name of the language is an in-joke, indicating that C-- is a reduced form of C, in the same way that "C++" was chosen to connote an improved version of C. (In C, -- and ++ mean "decrement" and "increment", respectively.)^[8]

Work on C-- began in the late 1990s. Since writing a custom code generator is a challenge in itself, and the compiler backends available to researchers at that time were complex and poorly documented, several projects had written compilers which generated C code (for instance, the original Modula-3 compiler). However, C is a poor choice for functional languages: it does not guarantee tail-call optimization, or support accurate garbage collection or efficient exception handling. C-- is a tightly defined simpler alternative to C which supports all of these. Its most innovative feature is a run-time interface which allows writing of portable garbage collectors, exception handling systems and other run-time features which work with any C-- compiler.

The first version of C-- was released in April 1998 as a MSRA paper,^[1] accompanied by a January 1999 paper on garbage collection.^[2] A revised manual was posted in HTML form in May 1999.^[9] Two sets of major changes proposed in 2000 by Norman Ramsey ("Proposed Changes") and Christian Lindig ("A

C--	
	
Unofficial logo for C-- which closely resembles the <u>C++</u> logo	
Paradigm	<u>imperative</u>
Designed by	<u>Simon Peyton Jones</u> and <u>Norman Ramsey</u>
First appeared	1998
Typing discipline	<u>static</u> , <u>weak</u>
Website	<u>www.cs.tufts.edu/~nr/c--/</u> (<u>http://www.cs.tufts.edu/~nr/c--/</u>)
Influenced by	
<u>C</u>	

New Grammar") led to C-- version 2, which was finalized around 2004 and officially released in 2005.^[3]

Type system

The C-- type system is designed to reflect constraints imposed by hardware rather than conventions imposed by higher-level languages. A value stored in a register or memory may have only one type: bit-vector. However, bit-vector is a polymorphic type which comes in several widths, e.g. `bits8`, `bits32`, or `bits64`. A separate 32-or-64 bit family of floating-point types is supported. In addition to the bit-vector type, C-- provides a boolean type `bool`, which can be computed by expressions and used for control flow but cannot be stored in a register or memory. As in an assembly language, any higher type discipline, such as distinctions between signed, unsigned, float, and pointer, is imposed by the C-- operators or other syntactic constructs. C-- is not type-checked, nor does it enforce or check the calling convention.^{[3]:28}

C-- version 2 removes the distinction between bit-vector and floating-point types. These types can be annotated with a string "kind" tag to distinguish, among other things, a variable's integer vs float typing and its storage behavior (global or local). The former is useful on targets that have separate registers for integer and floating-point values. Special types for pointers and the native word were introduced, although they are mapped to a bit-vector with a target-dependent length.^{[3]:10}

Example code

The following C-- code calculates the sum and product of integers 1 through n ^[10] (n is received as an argument). It demonstrates two language features:

- Procedures can return multiple results.
- Tail recursion is explicitly requested with the "jump" keyword.

```
/* Tail recursion */
export sp;
sp( bits32 n ) {
  jump sp_help( n, 1, 1 );
}

sp_help( bits32 n, bits32 s, bits32 p ){
  if n==1 {
    return( s, p );
  } else {
    jump sp_help( n-1, s+n, p*n );
  }
}
```

Implementations

The specification page of C-- lists a few implementations of C--. The "most actively developed" compiler, Quick C--, was abandoned in 2013.^[11]

Haskell

Some developers of C--, including Simon Peyton Jones, João Dias, and Norman Ramsey, work or have worked on GHC, whose development has led to extensions in the C-- language, forming the *Cmm* dialect which uses the C preprocessor for ergonomics.^{[4][12]}

GHC backends are responsible for further transforming C-- into executable code, via LLVM IR, slow C, or directly through the built-in native backend.^{[13][14][15]} Despite the original intention, GHC does perform many of its generic optimizations on C--. As with other compiler IRs, the C-- representation can be dumped for debugging.^[16] Target-specific optimizations are performed later by the backend.

Processing systems

As of 2018, most processing systems are not maintained, nor is their source code released.

- **Quick C--** is a compiler developed by The Quick C-- Team. It compiles version 2 of C-- code to Intel x86 Linux machine code. Compilation to machine code for other platforms is available as an experimental feature. Previously, Quick C-- was developed in parallel with the evolution of the C-- language specification, but the project was archived in 2019 on GitHub and development has ceased, though the source code is available there.
- **cmmc** is a C-- compiler implemented in the ML programming language by Fermin Reig. It generates machine code for Alpha, Sparc, and x86 architectures.^[17]
- **Trampoline C-- Compiler** is a C-- to C transpiler developed by Sergei Egorov in May 1999. It translates C-- code into C code, allowing it to be compiled using standard C compilers.
- **The Oregon Graduate Institute's C-- compiler** (OGI C-- Compiler) is the earliest prototype C-- compiler, developed in 1997 using the ML programming language. Maintenance of the OGI C-- Compiler was discontinued once development of Quick C-- began.

See also



- BCPL
- LLVM

References

1. Nordin, Thomas; Jones, Simon Peyton; Iglesias, Pablo Nogueira; Oliva, Dino (1998-04-23). "The C-- Language Reference Manual" (<https://www.microsoft.com/en-us/research/publication/the-c-language-reference-manual/>). *Microsoft*.
2. Reig, Fermin; Ramsey, Norman; Jones, Simon Peyton (1999-01-01). "C--: a portable assembly language that supports garbage collection" (<https://www.microsoft.com/en-us/research/publication/portable-assembly-language-supports-garbage-collection/>). *Microsoft*: 1–28.
3. Ramsey, Norman; Jones, Simon Peyton. "The C-- Language Specification, Version 2.0" (<http://www.cs.tufts.edu/~nr/c--/extern/man2.pdf>) (PDF). Retrieved 11 December 2019.

4. [GHC Commentary: What the hell is a .cmm file?](https://gitlab.haskell.org/ghc/ghc/wikis/commentary/rts/cmm) (<https://gitlab.haskell.org/ghc/ghc/wikis/commentary/rts/cmm>)
5. "An improved LLVM backend" (<https://ghc.haskell.org/trac/ghc/wiki/ImprovedLLVMBackend>). April 2019.
6. Oliva, Dino; Nordin, T.; Peyton Jones, Simon (1997-01-01). "C-: A Portable Assembly Language" (<https://www.microsoft.com/en-us/research/publication/c-a-portable-assembly-language/>). *Proceedings of the 1997 Workshop on Implementing Functional Languages* – via Microsoft.
7. Jones, Simon Peyton; Nordin, Thomas; Oliva, Dino (1998). "C--: A portable assembly language" (<https://link.springer.com/chapter/10.1007/BFb0055421>). In Clack, Chris; Hammond, Kevin; Davie, Tony (eds.). *Implementation of Functional Languages*. Lecture Notes in Computer Science. Vol. 1467. Berlin, Heidelberg: Springer. pp. 1–19. doi:10.1007/BFb0055421 (<https://doi.org/10.1007%2FBFb0055421>). ISBN 978-3-540-68528-9.
8. "Increment And Decrement Operators In C With Precedence" (<https://unstop.com/blog/increment-and-decrement-operators-in-c>). *unstop.com*. Retrieved 2024-06-20.
9. Nordin, Thomas; Jones, Simon Peyton; Iglesias, Pablo Nogueira; Oliva, Dino (1999-05-23). "The C– Language Reference Manual" (<https://www.cs.tufts.edu/~nr/c--/extern/manual.html>).
10. Ramsey, Norman; Jones, Simon Peyton; Lindig, Christian (2005-02-23), *The C-- Language Specification, version 2.0 (CVS Revision 1.128)* (<https://www.cs.tufts.edu/~nr/c--/extern/man2.pdf>) (PDF), p. 7, retrieved 2023-06-22
11. "C-- Downloads" (<https://www.cs.tufts.edu/~nr/c--/code.html>). *www.cs.tufts.edu*. Retrieved 11 December 2019.
12. "5.10. GHC Backends — Glasgow Haskell Compiler 9.8.1 User's Guide" (https://downloads.haskell.org/ghc/9.8.1/docs/users_guide/codegens.html#:~:text=5.10.,GHC%20Backends,compiling%20it%20to%20executable%20code.). *downloads.haskell.org*. Retrieved 2024-06-20.
13. [GHC Backends](https://downloads.haskell.org/~ghc/7.8.3/docs/html/users_guide/code-generators.html) (https://downloads.haskell.org/~ghc/7.8.3/docs/html/users_guide/code-generators.html)
14. "Opinion piece on GHC backends" (<https://andreaspk.github.io/posts/2019-08-25-Opinion%20piece%20on%20GHC%20backends.html>). *andreaspk.github.io*. August 25, 2019. Retrieved 2024-06-20.
15. "Using the Glasgow Haskell Compiler (GHC)" (<https://programmingdoer.com/>). *ProgDoer*. Retrieved 2024-06-20.
16. [Debugging compilers with optimization fuel](http://blog.ezyang.com/2011/06/debugging-compilers-with-optimization-fuel/) (<http://blog.ezyang.com/2011/06/debugging-compilers-with-optimization-fuel/>)
17. "C-- Downloads" (<https://www.cs.tufts.edu/~nr/c--/code.html>). *www.cs.tufts.edu*. Retrieved 2024-06-20.

External links

- [Archive of old official website](https://web.archive.org/web/20080822062234/http://www.cminusminus.org/) (<https://web.archive.org/web/20080822062234/http://www.cminusminus.org/>) (*cminusminus.org*)
 - [Quick C-- code archive](https://github.com/nrnrrn/qc--) (<https://github.com/nrnrrn/qc-->) (the reference implementation)
-

Retrieved from "<https://en.wikipedia.org/w/index.php?title=C--&oldid=1305115378>"